

LogPress Phase 1

Field-Role 기반 Line-to-Line Attention 분석

Thunderbird 로그만 입력해 self-attention 관계를 측정하고, 그 신호가 실제 중요 로그 보존에 유효한지 검증하는 1 단계 설계

Logs only

Field-aware

Line-to-Line

KV retention

핵심 전제

중요 로그가 높은 attention 을 받을지는 아직 모른다. 먼저 측정하고 검증한 뒤 보존 정책으로 연결한다.

연구 목표와 전체 흐름

RESEARCH QUESTION

무엇을 확인하려는가?

로그 자체의 self-attention 에서 어떤 field/role 이 line 간 관계를 만들고 ,
그 관계가 실제 중요 line 과 연결되는지 확인한다 .

DO NOT ASSUME

처음부터 가정하지 않는 것

높은 attention = 중요 line, Semantic 이 항상 최우선 , attention = causal
importance 라고 두지 않는다 .

01 Logs only

Thunderbird structured logs

02 Token attention

Qwen self-attention 추출

03 Field / Role

token → field → role

04 Line-to-Line

role 별 관계 matrix

05 Validation

label · layer · perturbation

06 KV retention

검증된 신호만 보존에 사용

Thunderbird 단계는 RCA 성능 증명이 아니라 attention 기반 진단 신호가 존재하는지 확인하는
mechanism study

Thunderbird Field → Role — 이름보다 정보의 역할로

Semantic

content

무엇이 발생했는가 (event/message)

Source

component

어디서 기록했는가 (process/service)

Entity

user • location

어느 node/entity 인가 (provenance)

Temporal

canonical time

언제 발생했는가 (token signal 분리)

EXCLUDED FROM IMPORTANCE

Position / Ground Truth

line_id 는 식별 • 순서용이며 field importance 계산에서 제외한다 . label 은 alert/non-alert ground truth 이므로 모델 입력에 넣지 않는다 .

TEMPORAL CLEANUP

시간 표현은 하나만 사용

date/month/day/time/timestamp 를 모두 넣지 않는다 . LLM 엔 canonical time 하나만 , numeric timestamp 는 구조 관계 계산에 사용한다 .

초기 prior 는 역할의 ' 기능 ' 만 정의한다 . 어떤 role 이 더 중요한지는 실험 결과로 결정하며 사전 weight 는 주지 않는다 .

Field 중요도 계산 — 평균을 메인, 합은 보조로

Line i 의 token 들이 Line j 의 특정 field 를 얼마나 보는지 계산한다.

MAIN SCORE

Mean / token

해당 field token attention 을 모두 모은 뒤 token 수로 나눈 평균 . content 처럼 긴 field 가 token 수만으로 유리해지는 문제를 줄인다 .

AUXILIARY SCORE

Attention Mass

해당 field 전체에 배분된 attention 총량 . 모델 attention budget 이 어느 field 에 많이 배분되는지 확인하는 보조 지표 .

계산 흐름

① Line i token

query token 집합

② Line j field

time/user/location/component/content

③ Attention 수집

query → 해당 field token

④ Field score

Mean/token + Mass 저장

content → Semantic • component → Source • user+location → Entity • time → Temporal-token

메인 실험 — Role-aware Line-to-Line

질문 없이 로그만 입력하고, 각 line 의 token 이 이전 line 의 어떤 field 를 바라보는지 측정한다.

Line 0	time user location component content
Line 1	time user location component content
Line 2	time user location component content
Line 3	time user location component content

예: Line 3 이 Line 2·Line 1 을 어떻게 참조하는지 확인 (causal LLM — 현재가 이전을 바라봄)

ONE RELATION

한 line 관계를 role 별로 분해

Semantic·Source·Entity·Temporal·token 점수를 각각 유지한다. 초기 단계에선 하나의 최종 점수로 합치지 않는다.

Role 별 Line-to-Line 관계

Line 3 → Line 2	강한 관계
Line 3 → Line 1	중간 관계
Line 2 → Line 1	약한 관계

검증 — 높은 attention 이 정말 유용한가 ?

VALIDATION

Layer / Head

Early·Middle·Late· 전체 평균으로 나눠 같은 role score 가 유지되는지 확인 . 1 차는 head 평균 .

VALIDATION

Temporal

시간 문자열 attention 과 실제 line 간 시간 근접성은 별도 신호 . 낮은 timestamp attention 이 곧 시간 정보 불필요를 뜻하진 않는다 .

VALIDATION

Perturbation

content 를 가리거나 component/user/location/timestamp 를 shuffle 해 line-field 연결을 깨고 성능 변화를 본다 .

COMPARE PERFORMANCE

Perturbation 기여도 해석

원본 성능과 특정 role 을 제거 · 섞은 뒤 성능 차이를 비교한다 . 차이가 클수록 그 role 이 판단에 실제 기여했을 가능성이 높다 .

ALIGNMENT

attention 과 실제 기여가 같이 높은가 ?

attention 도 높고 perturbation 시 성능도 크게 떨어지면 , 그 role 의 attention signal 이 실제 relevance 와 정렬된다는 근거가 강해진다 .

검증 전에는 attention score 를 곧바로 KV 보존 점수로 사용하지 않는다 .

최종 score 와 가설이 틀렸을 때

WEIGHTING

Role weight 는 사전에 정하지 않음

처음엔 모든 role 을 동일하게 관찰 . perturbation·relevance 결과가 확인된 뒤 , 성능 기여가 큰 role 에 더 큰 비중을 준다 .

FINAL RELATION

최종 Line-to-Line score

검증 후에만 role 별 relation 을 실험 기반 비중으로 결합 . Temporal-relation 은 별도 구조 신호로 유지하다 마지막에 결합 .

가설 결과에 따른 다음 단계

정렬됨

- important line 이 반복적으로 높은 score
- → role-aware KV retention 으로 연결

부분 정렬

- 특정 layer/head/role 에서만 신호
- → aggregation 방식을 더 세분화

정렬 안 됨

- raw attention 과 relevance 불일치
- → same source/entity/time 등 structural relation 결합

Phase 1 요약 — 현재 구현 우선순위

① Token mapping

각 token 을 정확한 field 와 line 에 매핑

② Field score

Mean/token 메인 + Mass 보조

③ Role matrix

Semantic/Source/Entity/Temporal-token

④ Line-to-Line

질문 없이 self-attention 관계 계산

⑤ Validation

label·layer·perturbation 으로 검증

⑥ KV 연결

검증된 경우에만 보존 정책으로 사용

핵심 — attention 이 높은 로그를 바로 보존하는 게 아니라 , 먼저 'attention 이 실제 중요 로그와 정렬되는지 ' 를 검증한다 .

LogPress Week 5

SnapKV Baseline 재현 완료

Qwen2.5-3B-Instruct · kvpress 0.5.4 · Thunderbird sample

이번 주 핵심 — SnapKV 압축 재현 → 50~90% 압축률 sweep → 원본 token index 추적 → line 별 KEEP/PARTIAL/DROP 및 원문 로그 출력까지 연결

StreamingLLM 다음 baseline · Week5 발표 요약 | 2026.08

SnapKV 재현 및 생존 토큰 추적

SnapKV 동작

1 Observation window

마지막 64 token 의 attention 사용

2 Score + pooling

kernel_size=5 로 중요도 smoothing

3 Top-k pruning

압축률에 맞춰 높은 score KV 만 보존

4 Tracking

레이어 / 헤드별 선택된 원본 token index 저장

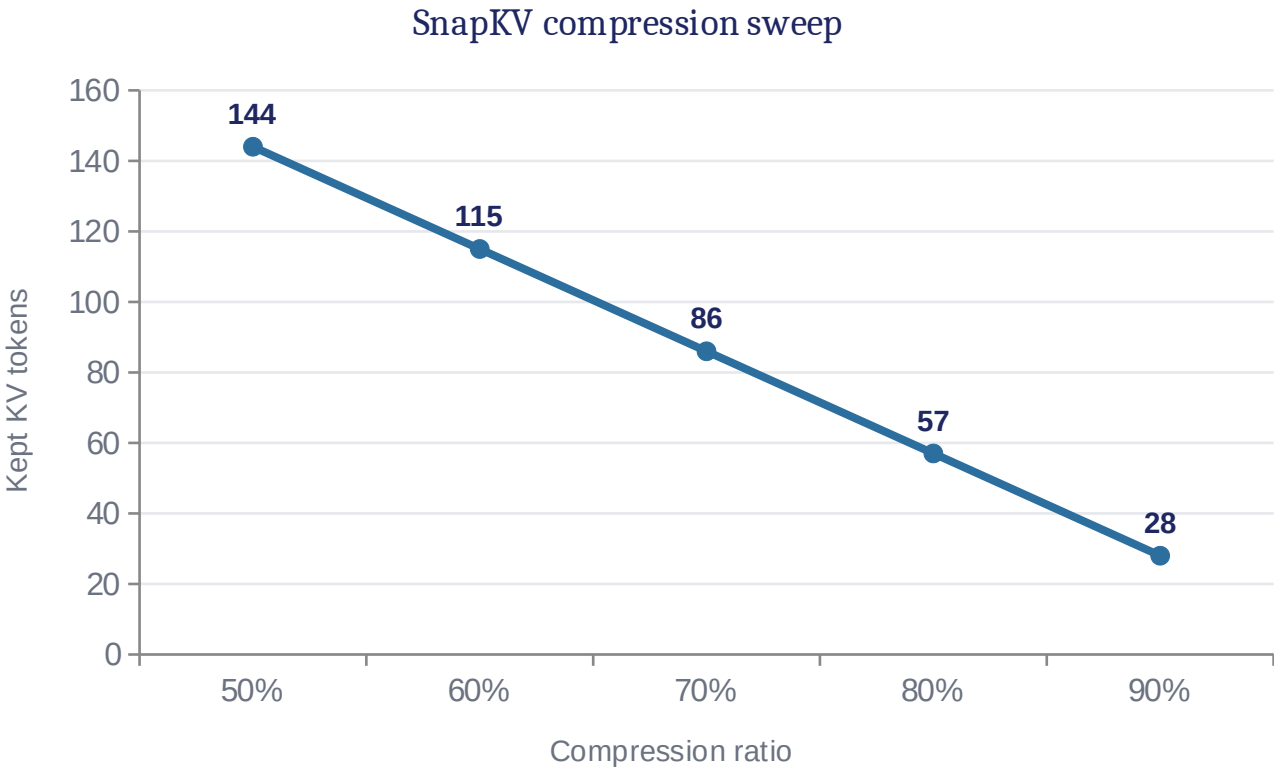
Attention 기반 baseline • 36 layers 추적

Token - Line 연결

- Tokenizer 에서 offset_mapping 확보
- 로그 30 개 line 의 문자 범위 line_spans 생성
- 289 개 token 을 각 로그 line 에 매핑
- TrackingSnapKVPress 로 top-k indices 저장
- line 별 생존 token 수 집계 → KEEP/PARTIAL/DROP
- 판정 결과와 함께 로그 원문 출력

압축률 Sweep 결과

입력 289 tokens • Qwen2.5-3B • 36 layers



압축률	남은 KV	확인 결과
50%	144	정상
60%	115	정상
70%	86	정상
80%	57	정상
90%	28	정상

검증 완료 — 첫 / 중간 / 마지막 layer 모두 동일 KV budget 적용

Week5 완료 사항 & 다음 실험

Week5 완료

- SnapKVPress 기반 prefill KV 압축 재현
- 압축률 50~90% sweep 확인
- 레이어·KV head 별 선택 token index 추적
- offset_mapping 기반 token-line 매핑
- 라인별 생존 token 수 및 KEEP/PARTIAL/DROP 출력
- 원문 로그와 생존 상태 함께 확인하는 디버그 출력

다음 실험 — PyramidKV

- 동일 Qwen / Thunderbird 입력 환경 유지
- PyramidKVPress 재현 및 KV shape 검증
- SnapKV와 동일하게 50~90% 압축률 sweep
- 동일 token-line 생존 추적 포맷으로 정리
- 이후 baseline 간 압축률 - 보존 특성 비교로 연결

진행 StreamingLLM ✓ SnapKV ✓ PyramidKV ▶

무엇을, 어디까지 할 수 있는가

결론을 앞세우기보다, 지금 되는 것과 안 되는 것의 경계를 정직하게 그린다. 이 경계가 이 발표의 뼈대다.

지금 할 수 있는 것

현재 데이터셋으로 바로

- 보존율 곡선 (proxy·baseline 4 종)
- 신호 ablation · keep_mask 근거
- 템플릿 홀드아웃 · 재현성 동결

지금은 못 하는 것

구조적으로 막힘

- 진짜 '원인' 보존율 (라벨 없음)
- 분산형 · 비국소 원인 검증
- 실시간 / 멀티홉 사슬 보존

조건 갖추면 되는 것

환경 제약만 풀리면

- Nezha 전처리 → 2 층 RCA
- 2 차 backbone · GPU 스왑
- floor 회피 추론모델

미래 과제로 남김

프로젝트 범위 밖

- 실시간 스트리밍 보존
- 학습 기반 KV
- 보존의 운영 가치

바닥 전제 현재 데이터셋 한정 — Thunderbird(이상 proxy 라벨) + Nezha(원인 라벨이나 소규모). 이 한 가지가 위 경계 대부분을 만든다.

왜 여기서 선을 긋는가

모든 경계의 원인 = 데이터셋

Thunderbird — 표본은 많지만 '이상' 라벨 (원인 아님) → 1 층은 proxy 까지 . **Nezha** — 진짜 '원인' 라벨이지만 소규모 · 저부하 → 2 층의 유일한 근거이나 대표성 한계 .

같은 축인가, 갈라지는가

기준 하나 — 원 축 (KV 압축으로 원인 보존) 을 유지하면 '안', 축이 바뀌면 '밖' .

프로젝트 '안' — 이어갈 것

같은 축의 자연스러운 다음 챕터

- Nezha 원인 라벨로 보존 → RCA 검증
- 희귀도 재설계 (라벨 누수 끊기)
- keep_mask 설명가능성

프로젝트 '밖' — 갈라지는 새 문제

비슷해 보여도 축이 다른 연구

- 실시간 스트리밍 보존 (설계 변경)
- 다른 로그 태스크용 KV 압축
- 학습 기반 KV · 보존의 운영 가치

정직한 수위 "보존이 RCA 를 향상시킨다" 가 아니라 — 저장 로그의 사후 분석에 한정해 , 보존과 성능의 관계를 처음으로 쟁다 .

McNemar — 두 방법 중 누가 원인을 더 남겼나

1 무엇을 하나 — 2×2 로 놓고 본다

	baseline 보존	baseline 제거
LogPress 보존	a	b
LogPress 제거	c	d

회색 (a·d) 같은 판정 → 버림 **주황 (b·c)** 엇갈림 → 이것만 씀

두 보존율의 차이 = $(b - c) / n$ — a·d 는 상쇄되어 안 들어감

숫자로

a=800, d=50 은 버리고 b=90, c=40 만 사용

$\chi^2 = (90-40)^2 / (90+40) \approx 19.2 \rightarrow p \approx 0.00001$ (유의)

b(90) > c(40) → LogPress 가 더 많이 보존

2 핵심 원리 — 엇갈린 것만 동전 던지기

둘이 실력이 같다면, 엇갈린 쌍이 어느 쪽으로 갈 확률은 반반이어야 한다. → 불일치쌍 b+c 개를 **공정한 동전 b+c 번 던지기**로 보고, b 가 c 보다 유의하게 쏠렸는지 확인한다.

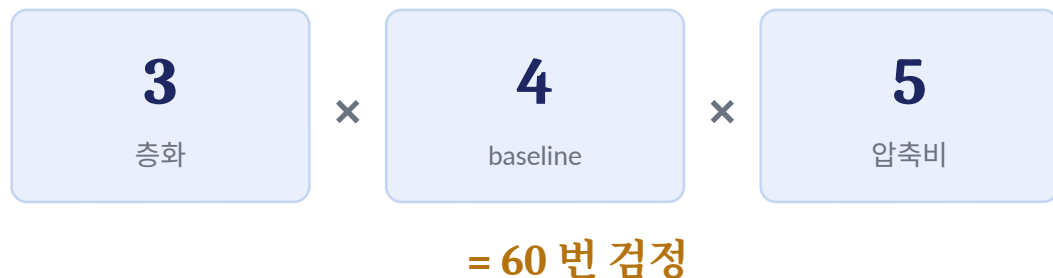
? 내가 부딪힌 질문

Q. "비교 검정" 이라며? 그런데 왜 '우연일 확률' 을 계산하지?

A. 먼저 "두 방법이 같다" 고 가정하고, 90:40 같은 치우침이 우연으로 나올 확률을 잰다. 그 확률이 너무 작으면 "같다" 는 가정을 접는다 = 두 방법이 다르다. 확률을 재는 것이 곧 비교하는 방법이다.

BH 보정 — 60 번 검정한 결과를 어떻게 믿나

1 왜 보정하나 — 여러 번 하면 우연이 쌓인다



60 개 전부 진짜 차이가 없어도 $60 \times 0.05 =$ **평균 3 개**가 우연히 '유의'로 뜬다. 그래서 문턱을 조여야 한다.

Tier-1 판정식

하드케이스 (alert 태그 + 본문 non-ERROR) 에서
4 개 baseline 모두 대비
 $p_{bh} < 0.05$ AND $b > c$ 이면 통과

2 원리 — p 값을 재료로 문턱을 조정

p 값 60 개를 줄 세워, 순위별로 합격선을 조정한다. 통제하는 것은 "**고른 것 중 가짜 비율 (FDR)**" 을 5% 이하로. Bonferroni 보다 덜 엄격해 검정력이 높다.

"p 값을 재료로" = 채점 (McNemar) 은 끝났고, BH 는 그 점수표만 보고 합격선을 정하는 감독관. 원자료는 안 본다.

? 내가 부딪힌 질문

Q. 60 개 검정이 서로 얹혀 있는데, BH 를 그냥 써도 되나?

A. 60 개가 전부 LogPress 를 공유해 '같은 방향으로' 얹힌다 (양의 상관). 이 경우 BH 는 오히려 더 보수적으로 작동해 이미 안전하다. 다만 완전히 증명하진 못하므로, 얹힘과 무관하게 보장되는 Holm 을 함께 돌려 방어한다.